

STEP 4

Model training, comparison and quality gate

Train several candidates, pick the best on the right metric, and refuse to ship a weak model.

What happens when you run training

1. Load and validate the CSV.
2. Stratified 80/20 split (6,400 train / 1,600 test). Stratified means each split keeps the 54/30/16 class mix.
3. Train three pipelines: Logistic Regression, Random Forest, HistGradientBoosting.
4. Score each on the test set with **macro F1** and accuracy; pick the best by macro F1.
5. **Quality gate:** if the best macro F1 is below 0.80, stop with exit code 1 and save nothing.
6. Otherwise save the model, metrics.json, the confusion matrix and a snapshot of the training data.

```
python -m foodwaste.train
```

```
logistic_regression    macro F1 0.8236    accuracy 0.8562
random_forest          macro F1 0.8451    accuracy 0.8738
hist_gradient_boosting macro F1 0.8648    accuracy 0.8900
```

Best model: hist_gradient_boosting (macro F1 0.8648)

Quality gate passed (≥ 0.8). Saved model version 20261004-093839 to models/model.joblib

Results

Class	Precision	Recall	F1	Test rows
Low	0.94	0.94	0.94	866
Medium	0.82	0.86	0.84	480
High	0.85	0.78	0.81	254
Macro average	0.87	0.86	0.86	1,600

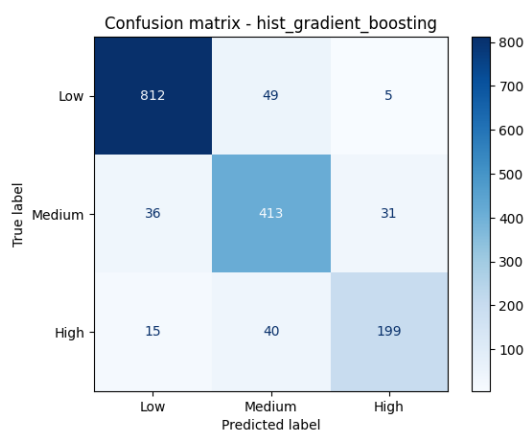


Figure 3 - Confusion matrix of the best model on the 1,600 test rows. Most errors are between neighbouring levels (High vs Medium).

Why macro F1 and not accuracy?

A model that always answers 'Low' already scores 54% accuracy while being useless. **Macro F1** averages the F1 score of each class equally, so it only goes up if the model also gets Medium and High right. A test also checks that accuracy beats the always-Low baseline.

Artifacts written to models/

File	Contents	Used by
model.joblib	The full fitted pipeline	API, UI, notebook
metrics.json	Version, scores of every candidate, per-class report, data MD5 hash, sklearn version, feature list	API /model-info, UI Model tab
confusion_matrix.png	Chart above	UI Model tab
reference_data.csv	The training rows	Drift monitoring baseline

Why it matters

The quality gate turns 'we hope the new model is good' into an automatic rule. In CI and in docker build, a model below the threshold makes the whole build fail, so it can never be deployed.